

A. 排列题 (permutation)

考虑如果 n 是偶数，那么每种颜色都得出偶数次，直接中心对称的构造即可，

如果 n 是奇数，则至多有一个颜色是出现了一次，总共有奇数个颜色出现了奇数次。

那么我们选择那个出现一次的，或者总共出现奇数次的颜色，放一个棋子在中间。

接下来，我们的目标是把所有出现次数为奇数 (≥ 3 次) 的转换为偶数，最好还使得剩余的格子仍然对称。

令有 k 个出现次数为偶数的，那么将他们两两配对得到 cnt 对，经过手玩，一种构造为： $mid + i$, $mid + i + cnt$, $mid - 2i - cnt$; $mid - i$, $mid - 3cnt + 1 + 2i$, $mid + 3cnt - 1 - i$ 。

复杂度： $O(n)$ 。

B. 数数题 (count)

考虑如何判定一个序列是否是好的。

首先显然是定义权值为“选则 + 不选择则 -”，然后如果一个区间的最大独立集权值为 0，则不合法。

那么容易想到，记录每个前缀的最大后缀独立集权值和（两个信息，一个是最后一个位置选的，另一个是最后一个位置不选的）。

我们令最后一个随便/不选的权值分别为 a, b ，不合法当且仅当某一个 $a = 0$ 。

注意到，如果某个时刻 $a + b > 0$ ，那么显然后面永远都会 $a + b > 0$ 。而 $a \geq b$ ，所以之后永远 $a > 0$ ，直接合法。

而 $a + b \geq 0$ ，所以只有 $a + b = 0$ 的情况是有意义的。

对于状态 $(a, -a)$ ，在后面加入一个数 x ，得到的新权值为： $\max\{x - a, a - x\}, a - x$ 。

若 $x < a$ ，那么直接合法，若 $x = a$ ，那么直接不合法，若 $x > a$ 则变为状态 $x - a, a - x$ ，也就是 a 变为 $x - a$ 。

于是判定序列 a 的所有前缀是否合法的方式为：令 $s_i = a_i - s_{i-1}$ ，不合法当且仅当存在 $s_i = 0$ ，且第一个 $s_i = 0$ 的位置前面的都满足 $s_i > 0$ 。

那你需要判定序列 a 的所有区间是否合法，就只需要考虑前缀的后缀，也就是对每个前缀，维护其所有后缀跑这个过程对应终止的 a 的值（如果已经确定合法就不管了）

复杂度： $O(nm2^m)$ 。

现在要优化的东西，形如：

```
dp[i][rev[S+1]>>(m-j)] += dp[i-1][S]
```

对于一个固定的 j ，涉及的 $dp[i]$ 的修改为 2^j 次，对应的每轮每次修改产生贡献可以快速 $O(2^m)$ 预处理。

因此总复杂度可以优化为 $O(n2^m)$ 。

C. 最短路问题 (shortest)

我们考虑 $s = 0, t = n - 1$ 怎么做。

1. 首先，不会走回头路，也就是横向路程固定。
2. 尽量减少“桥的切换”。
3. 对于高桥低桥切换：与高桥有交点的，一定可以切换到低桥，但是反之则不行。

于是我们希望尽可能保持在高桥，也就是说：可以调整，使得每次上高桥都是从高桥起点处上去，从高桥下来则都是从高桥终点处下来。

我们把所有桥的起点终点称为关键点。

进一步观察，可以得到：

每次上高桥或下高桥的时候，都不会经过非关键点的交点。

因为如果经过，就要么可以从那个桥的起点处上来，要么就直接下降到那个桥上面去。

于是关键点有 $O(m)$ 个，连边也是关键点下面第一个交点对应的桥可以走到那里。

那么点数边数就都是 $O(m)$ 了，直接跑 Dijkstra 即可。

对于一般的问题，我们考虑会发生的变化。

首先就是可能会走回头路了，但是你会发现，走回头路的原因是想要走高桥，而肯定是在能上高桥的第一个交点处就直接上去了。

于是我们对一个桥，建立至多六个关键点： l, r, s 左侧第一个， s 右侧第一个， t 左侧第一个， t 右侧第一个。

之后还是有类似的结论——每次上高桥或下高桥的时候，都不会经过非关键点的交点。

以上高桥为例，如果是上到高桥 s 左侧第一个交点，那么经过的途径交点要么是一个桥 s 左侧第一个交点，要么就之前存在这样一个交点，可以在第一次相交的位置上上去。

于是还是对所有关键点以及关键点下第一个交点建点，之后跑 Dijkstra。

复杂度 $O(m \log m)$ 。